

CTE Tasks and Solutions

```
USE SalesDB;
```

```
-----  
-- the one restrict in CTE is can not use ORDER BY clause --  
-----
```

```
-- find total sales per customer
```

```
WITH CTE_Total_Sales AS  
(  
    SELECT  
        CustomerID,  
        SUM(Sales) AS TotalSales  
    FROM Sales.Orders  
    GROUP BY CustomerID  
)  
SELECT  
    c.CustomerID,  
    c.FirstName,  
    c.LastName,  
    cts.TotalSales  
FROM Sales.Customers AS c  
LEFT JOIN CTE_Total_Sales AS cts  
ON c.CustomerID = cts.CustomerID  
ORDER BY c.CustomerID DESC;
```

```
-----  
-- when use multiple CTEs, use comma to separate CTEs --  
-----
```

```
-- Step 1: Find total sales per customer
```

```
-- Step 2: Find the last order date per customer
```

```
WITH CTE_Total_Sales AS  
(  
    SELECT  
        CustomerID,  
        SUM(Sales) AS TotalSales  
    FROM Sales.Orders  
    GROUP BY CustomerID  
)  
CTE_Last_Order AS  
(  
    SELECT  
        CustomerID,  
        MAX(OrderDate) AS LastOrderDate  
    FROM Sales.Orders  
    GROUP BY CustomerID  
)  
SELECT  
    c.CustomerID,  
    c.FirstName,  
    c.LastName,  
    cts.TotalSales,  
    clo.LastOrderDate  
FROM Sales.Customers AS c  
LEFT JOIN CTE_Total_Sales AS cts
```

```
    ON c.CustomerID = cts.CustomerID
LEFT JOIN CTE_Last_Order AS clo
    ON c.CustomerID = clo.CustomerID
ORDER BY cts.TotalSales DESC;
```

-- Create a CTE that calculates the total sales for each customer, then display the customer name and total sales.

```
WITH CTE_Total_Sales AS
(
    SELECT
        CustomerID,
        SUM(Sales) AS TotalSales
    FROM Sales.Orders
    GROUP BY CustomerID
)
SELECT
    c.CustomerID,
    c.FirstName,
    c.LastName,
    cts.TotalSales
FROM Sales.Customers AS c
LEFT JOIN CTE_Total_Sales AS cts
    ON c.CustomerID = cts.CustomerID;
```

-- Create a CTE that calculates the number of orders for each customer.

```
WITH CTE_Order_Count AS
(
    SELECT
        CustomerID,
        COUNT(*) AS TotalOrders
    FROM Sales.Orders
    GROUP BY CustomerID
)
SELECT
    c.CustomerID,
    c.FirstName,
    c.LastName,
    coc.TotalOrders
FROM Sales.Customers AS c
LEFT JOIN CTE_Order_Count AS coc
    ON c.CustomerID = coc.CustomerID;
```

-- Create a CTE that calculates the average order sales for each customer

```
WITH CTE_Average_Sales AS
(
    SELECT
        CustomerID,
        AVG(Sales) AS AverageSales
    FROM Sales.Orders
    GROUP BY CustomerID
)
SELECT
    c.CustomerID,
    c.FirstName,
    c.LastName,
    cas.AverageSales
FROM Sales.Customers AS c
LEFT JOIN CTE_Average_Sales AS cas
```

```

    ON c.CustomerID = cas.CustomerID;

-- Create a CTE that finds the most recent order date for each customer.
WITH CTE_Last_Order AS
(
    SELECT
        CustomerID,
        MAX(OrderDate) AS LastOrderDate
    FROM Sales.Orders
    GROUP BY CustomerID
)
SELECT
    c.CustomerID,
    c.FirstName,
    c.LastName,
    clo.LastOrderDate
FROM Sales.Customers AS c
LEFT JOIN CTE_Last_Order AS clo
    ON c.CustomerID = clo.CustomerID;

-- Create a CTE that finds the first order date for every customer.
WITH CTE_First_Order AS
(
    SELECT
        CustomerID,
        MIN(OrderDate) AS FirstOrderDate
    FROM Sales.Orders
    GROUP BY CustomerID
)
SELECT
    c.CustomerID,
    c.FirstName,
    c.LastName,
    cfo.FirstOrderDate
FROM Sales.Customers AS c
LEFT JOIN CTE_First_Order AS cfo
    ON c.CustomerID = cfo.CustomerID;

-- Create a CTE containing total sales per customer, then return only customers whose
total sales are greater than 100
WITH CTE_Total_Sales AS
(
    SELECT
        CustomerID,
        SUM(Sales) AS TotalSales
    FROM Sales.Orders
    GROUP BY CustomerID
)
SELECT
    c.CustomerID,
    c.FirstName,
    c.LastName,
    cts.TotalSales
FROM Sales.Customers AS c
JOIN CTE_Total_Sales AS cts
    ON c.CustomerID = cts.CustomerID
WHERE cts.TotalSales > 100
ORDER BY cts.TotalSales DESC;

```

-- Create a CTE that counts orders per customer and return customers with at least 2 orders.

```
WITH CTE_Order_Count AS
(
    SELECT
        CustomerID,
        COUNT(*) AS TotalOrders
    FROM Sales.Orders
    GROUP BY CustomerID
)
SELECT
    c.CustomerID,
    c.FirstName,
    c.LastName,
    coc.TotalOrders
FROM Sales.Customers AS c
JOIN CTE_Order_Count AS coc
    ON c.CustomerID = coc.CustomerID
WHERE coc.TotalOrders >= 2
ORDER BY coc.TotalOrders DESC;
```

-- Create a CTE containing the last order date per customer and return customers whose last order occurred after 2025-03-01.

```
WITH CTE_Last_Order AS
(
    SELECT
        CustomerID,
        MAX(OrderDate) AS LastOrderDate
    FROM Sales.Orders
    GROUP BY CustomerID
)
SELECT
    c.CustomerID,
    c.FirstName,
    c.LastName,
    clo.LastOrderDate
FROM Sales.Customers AS c
JOIN CTE_Last_Order AS clo
    ON c.CustomerID = clo.CustomerID
WHERE clo.LastOrderDate > '2025-03-01';
```

-- Create a CTE that finds the highest sales amount for each customer.

```
WITH CTE_Max_Sales AS
(
    SELECT
        CustomerID,
        MAX(Sales) AS MaxSales
    FROM Sales.Orders
    GROUP BY CustomerID
)
SELECT
    c.CustomerID,
    c.FirstName,
    c.LastName,
    cms.MaxSales
FROM Sales.Customers AS c
JOIN CTE_Max_Sales AS cms
```

```
    ON c.CustomerID = cms.CustomerID
ORDER BY cms.MaxSales DESC;
```

```
-- Customer sales summary
```

```
WITH CTE_Customer_Summary AS
```

```
(
    SELECT
        CustomerID,
        COUNT(*) AS TotalOrders,
        SUM(Sales) AS TotalSales,
        AVG(Sales) AS AverageSales,
        MAX(Sales) AS MaxSales,
        MIN(Sales) AS MinSales
    FROM Sales.Orders
    GROUP BY CustomerID
)
SELECT
    c.CustomerID,
    c.FirstName,
    c.LastName,
    ccs.TotalOrders,
    ccs.TotalSales,
    ccs.AverageSales,
    ccs.MaxSales,
    ccs.MinSales
FROM Sales.Customers AS c
LEFT JOIN CTE_Customer_Summary AS ccs
    ON c.CustomerID = ccs.CustomerID;
```

```
-- Total sales + last order
```

```
WITH CTE_Total_Sales AS
```

```
(
    SELECT
        CustomerID,
        SUM(Sales) AS TotalSales
    FROM Sales.Orders
    GROUP BY CustomerID
),
CTE_Last_Order AS
(
    SELECT
        CustomerID,
        MAX(OrderDate) AS LastOrderDate
    FROM Sales.Orders
    GROUP BY CustomerID
)
SELECT
    c.CustomerID,
    c.FirstName,
    c.LastName,
    cts.TotalSales,
    clo.LastOrderDate
FROM Sales.Customers AS c
LEFT JOIN CTE_Total_Sales AS cts
    ON c.CustomerID = cts.CustomerID
LEFT JOIN CTE_Last_Order AS clo
    ON c.CustomerID = clo.CustomerID;
```

-- Total sales + order count + last order

```
WITH CTE_Total_Sales AS
(
    SELECT
        CustomerID,
        SUM(Sales) AS TotalSales
    FROM Sales.Orders
    GROUP BY CustomerID
),
CTE_Order_Count AS
(
    SELECT
        CustomerID,
        COUNT(*) AS TotalOrders
    FROM Sales.Orders
    GROUP BY CustomerID
),
CTE_Last_Order AS
(
    SELECT
        CustomerID,
        MAX(OrderDate) AS LastOrderDate
    FROM Sales.Orders
    GROUP BY CustomerID
)
SELECT
    c.CustomerID,
    c.FirstName,
    c.LastName,
    cts.TotalSales,
    coc.TotalOrders,
    clo.LastOrderDate
FROM Sales.Customers AS c
LEFT JOIN CTE_Total_Sales AS cts
    ON c.CustomerID = cts.CustomerID
LEFT JOIN CTE_Order_Count AS coc
    ON c.CustomerID = coc.CustomerID
LEFT JOIN CTE_Last_Order AS clo
    ON c.CustomerID = clo.CustomerID;
```

-- Customers above average total sales

```
WITH CTE_Total_Sales AS
(
    SELECT
        CustomerID,
        SUM(Sales) AS TotalSales
    FROM Sales.Orders
    GROUP BY CustomerID
),
CTE_Average_Sales AS
(
    SELECT
        AVG(TotalSales) AS AverageTotalSales
    FROM CTE_Total_Sales
)
SELECT
    c.CustomerID,
    c.FirstName,
    c.LastName,
```

```

        cts.TotalSales
FROM Sales.Customers AS c
JOIN CTE_Total_Sales AS cts
    ON c.CustomerID = cts.CustomerID
CROSS JOIN CTE_Average_Sales AS cas
WHERE cts.TotalSales > cas.AverageTotalSales
ORDER BY cts.TotalSales DESC;

```

-- Top customer by total sales

```

WITH CTE_Total_Sales AS
(
    SELECT
        CustomerID,
        SUM(Sales) AS TotalSales
    FROM Sales.Orders
    GROUP BY CustomerID
)
SELECT TOP 1
    c.CustomerID,
    c.FirstName,
    c.LastName,
    cts.TotalSales
FROM Sales.Customers AS c
JOIN CTE_Total_Sales AS cts
    ON c.CustomerID = cts.CustomerID
ORDER BY cts.TotalSales DESC;

```

-- Top 3 customers

```

WITH CTE_Total_Sales AS
(
    SELECT
        CustomerID,
        SUM(Sales) AS TotalSales
    FROM Sales.Orders
    GROUP BY CustomerID
)
SELECT TOP 3
    c.CustomerID,
    c.FirstName,
    c.LastName,
    cts.TotalSales
FROM Sales.Customers AS c
JOIN CTE_Total_Sales AS cts
    ON c.CustomerID = cts.CustomerID
ORDER BY cts.TotalSales DESC;

```

-- Customer spending category

```

WITH CTE_Total_Sales AS
(
    SELECT
        CustomerID,
        SUM(Sales) AS TotalSales
    FROM Sales.Orders
    GROUP BY CustomerID
)
SELECT
    c.CustomerID,

```

```

        c.FirstName,
        c.LastName,
        cts.TotalSales,
        CASE
            WHEN cts.TotalSales >= 100 THEN 'High'
            WHEN cts.TotalSales >= 70 THEN 'Medium'
            ELSE 'Low'
        END AS CustomerCategory
FROM Sales.Customers AS c
JOIN CTE_Total_Sales AS cts
    ON c.CustomerID = cts.CustomerID
ORDER BY cts.TotalSales DESC;

```

-- Customer activity status

```

WITH CTE_Last_Order AS
(
    SELECT
        CustomerID,
        MAX(OrderDate) AS LastOrderDate
    FROM Sales.Orders
    GROUP BY CustomerID
)
SELECT
    c.CustomerID,
    c.FirstName,
    c.LastName,
    clo.LastOrderDate,
    CASE
        WHEN clo.LastOrderDate >= '2025-03-01'
            THEN 'Active'
        ELSE 'Inactive'
    END AS CustomerStatus
FROM Sales.Customers AS c
LEFT JOIN CTE_Last_Order AS clo
    ON c.CustomerID = clo.CustomerID;

```

-- Customer performance summary

```

WITH CTE_Customer_Summary AS
(
    SELECT
        CustomerID,
        COUNT(*) AS TotalOrders,
        SUM(Sales) AS TotalSales,
        AVG(Sales) AS AverageOrderValue,
        MAX(OrderDate) AS LastOrderDate
    FROM Sales.Orders
    GROUP BY CustomerID
)
SELECT
    c.CustomerID,
    c.FirstName,
    c.LastName,
    ccs.TotalOrders,
    ccs.TotalSales,
    ccs.AverageOrderValue,
    ccs.LastOrderDate,
    CASE
        WHEN ccs.TotalSales >= 100 THEN 'High'
        WHEN ccs.TotalSales >= 70 THEN 'Medium'
    END AS CustomerCategory

```



```
        ELSE 'Low'  
    END AS CustomerCategory  
FROM Sales.Customers AS c  
LEFT JOIN CTE_Customer_Summary AS ccs  
    ON c.CustomerID = ccs.CustomerID  
ORDER BY ccs.TotalSales DESC;
```